

CloudShield AI

28 — API Specification

Version 1.0 | Product-Level API & Service Contract Specification

CloudShield AI / CloudSpartanX

Document purpose: Define the external REST API, internal service contracts, resource model, lifecycle operations, authentication, authorization, validation, asynchronous jobs, error semantics, idempotency, pagination, filtering, versioning, auditability, and security requirements for CloudShield AI.

Canonical principle: APIs expose governed product capabilities. They do not bypass lifecycle, authorization, verification, or promotion controls.

1. Purpose and Scope

This specification converts the CloudShield AI domain, functional, state, security, AI, DAIL, and infrastructure specifications into implementation-oriented API contracts. It is product-level: DAIL is represented as an integrated subsystem and does not define the entire API surface.

- Define the public application API used by the CloudShield AI web UI and approved clients.
- Define resource-oriented endpoints and lifecycle operations.
- Define authentication, authorization, tenant isolation, and scope controls.
- Define synchronous and asynchronous operation behavior.
- Define API contracts for cloud connections, assets, findings, risk, dependencies, AI, remediation candidates, DAIL verification, approvals, promotion, evidence, reports, jobs, policies, and experiments.
- Define common request/response/error conventions.
- Define API security and operational requirements.
- Provide an MVP API surface that can evolve without breaking domain semantics.

2. API Design Principles

- Resource-oriented and explicit.
- Stable domain identifiers rather than display names.
- Lifecycle transitions are explicit operations, not arbitrary status updates.
- Server-side authorization is mandatory for every protected operation.
- Tenant and cloud-scope isolation is enforced server-side.
- Long-running operations return job resources rather than holding HTTP requests open indefinitely.
- Infrastructure-affecting operations are governed and auditable.
- AI outputs are represented as recommendations or candidates, never as trusted state.
- DAIL verification is invoked through explicit contracts and cannot be bypassed by client-supplied status.
- Promotion requires all required preconditions to be satisfied at the server.
- API responses expose freshness and provenance where state may be stale or derived.
- Backward-compatible evolution is preferred over breaking changes.

3. Architectural Position

The API layer sits between the product UI/clients and CloudShield AI domain/application services.

Client →API Gateway/Ingress →Authentication →Authorization/Tenant Context →Application Services →Domain Modules →Workers/Queues →Database/Object Storage/AWS/LLM Provider.

API Boundary	Responsibility	Must Not Do
External API	Expose governed product capabilities.	Bypass domain authorization or lifecycle.
Application service	Validate commands and orchestrate domain actions.	Contain provider-specific SDK assumptions.
Worker API/contract	Execute asynchronous jobs.	Accept arbitrary client privilege escalation.
AI service contract	Request explanations/recommendations.	Grant cloud mutation authority.
DAIL contract	Verify candidate/impact/state.	Treat client-provided verification as authoritative.
Promotion service	Execute approved governed change.	Accept unverified candidates.

4. API Protocol Baseline

Item	MVP Standard
Transport	HTTPS only.
Style	REST-oriented JSON API.
Base path	/api/v1
Content type	application/json unless explicitly documented.
Authentication	Short-lived bearer access tokens / configured identity provider.
Character encoding	UTF-8.
Time format	ISO 8601 / UTC timestamps.
Identifiers	Opaque stable IDs; UUID/ULID-compatible implementation acceptable.
Pagination	Cursor-based for collection APIs.
Errors	Structured machine-readable error envelope.
Idempotency	Supported for mutating operations where retries are plausible.
Correlation	Request/correlation ID on every request/response.
Versioning	Major version in URL; additive changes preferred.

5. API Base Resource Model

Every tenant-scoped resource is associated with an organization/tenant context and, where applicable, a cloud connection and cloud scope.

Resource	Purpose
Organization/Tenant	Top-level authorization and isolation boundary.
User/Principal	Authenticated actor identity.
CloudConnection	Connection metadata and authorization to a customer cloud.
CloudScope	Account/region/resource scope used by operations.

Resource	Purpose
Asset	Canonical cloud resource identity.
Observation	Observed cloud/configuration fact.
ConfigurationSnapshot	Point-in-time normalized configuration state.
ConfigurationDiff	Difference between snapshots.
Finding	Security or configuration issue.
RiskAssessment	Risk evaluation for an asset/finding/change.
DependencyEdge	Canonical relationship between assets.
ImpactAnalysis	Blast-radius assessment for a candidate/change.
AIRequest	Structured request to the AI subsystem.
AIRecommendation	Grounded explanation/recommendation.
RemediationCandidate	Proposed infrastructure/application change.
CandidateVersion	Immutable version of candidate content.
DAILVerificationRun	Deterministic verification execution.
ApprovalDecision	Governance decision.
Promotion	Governed execution record.
TrustedState	State earned after successful promotion.
EvidenceArtifact	Evidence associated with analysis/verification/promotion.
AuditEvent	Security and lifecycle audit record.
Job	Asynchronous operation.
Policy	Configurable governance/security policy.
Report	Generated product/reporting artifact.
Experiment/EvaluationRun	Controlled evaluation workload.

6. Authentication

- All protected API endpoints require authenticated access unless explicitly marked public.
- Access tokens must be short-lived and validated server-side.
- Token audience, issuer, expiry, signature, and required claims must be validated.
- The API must derive tenant/principal context from trusted authentication claims and server-side authorization data.
- Clients must not be allowed to select arbitrary tenant IDs as an authorization mechanism.
- Service-to-service calls use workload identity/service credentials appropriate to the deployment environment.
- Administrative endpoints require stronger role/permission checks.

7. Authorization and Tenant Isolation

7.1 Authorization model

Authorization is permission- and scope-based. A user may have product roles, but each sensitive operation must resolve the effective permission against the target resource and cloud scope.

Permission Class	Examples
Read	assets:read, findings:read, evidence:read.
Analyze	security:analyze, risk:assess, dependency:analyze.
AI	ai:request, ai:recommend.
Candidate	candidate:create, candidate:edit, candidate:submit.
Verify	verification:run, verification:read.
Approve	approval:grant, approval:reject.

Permission Class	Examples
Promote	promotion:execute.
Admin	connection:manage, policy:manage, user:manage.

7.2 Isolation rules

- Every query is tenant-scoped by server-side authorization.
- Resource IDs alone do not grant access.
- Cross-tenant references return a not-found or authorization-safe response without leaking existence.
- Cloud account/region/resource scopes are checked before cloud-sensitive operations.
- Evidence downloads require authorization to the underlying evidence/resource.
- Promotion authority cannot be inferred from ordinary candidate-edit permission.

8. Common Headers

Header	Requirement
Authorization	Bearer access token for protected endpoints.
Content-Type	application/json for JSON bodies.
Accept	Expected response media type.
Idempotency-Key	Required for designated retryable mutating operations.
X-Correlation-ID	Optional client-supplied correlation identifier; server validates/limits format.
If-Match	Optional optimistic concurrency control where supported.
If-None-Match	Optional cache/concurrency semantics for read resources.

The server returns a correlation/request identifier in responses so application, worker, audit, and infrastructure telemetry can be connected.

9. Common Resource Envelope

Single-resource responses should expose the resource directly or through a stable envelope. Collection responses should include data plus pagination metadata.

Field	Meaning
id	Stable resource identifier.
tenant_id	Returned only where safe and useful; never trusted from clients.
created_at	Creation timestamp.
updated_at	Last update timestamp.
version	Optimistic/resource version where applicable.
status	Current lifecycle state.
links	Optional navigational links.
provenance	Source/freshness metadata where applicable.

10. Error Contract

All application errors use a structured envelope.

Field	Example Meaning
code	RESOURCE_NOT_FOUND, VALIDATION_ERROR, FORBIDDEN.
message	Human-readable safe summary.
details	Field or rule-level structured details where appropriate.
request_id	Correlation identifier.
retryable	Whether retry may be appropriate.
timestamp	Server timestamp.

HTTP Status	Use
400	Malformed request or invalid syntax.
401	Missing/invalid authentication.
403	Authenticated but unauthorized.
404	Resource unavailable to caller.
409	Conflict, stale version, invalid state transition, or duplicate idempotency request.
422	Semantically invalid input.
429	Rate/usage limit exceeded.
500	Unexpected server error.
502/503/504	Controlled upstream/dependency/service failure.

Error messages must not expose secrets, cloud credentials, internal stack traces, provider tokens, or cross-tenant resource information.

11. Idempotency and Concurrency

- Mutating endpoints that create external side effects must support idempotency keys.
- The server stores idempotency outcome metadata long enough to prevent unsafe duplicate execution.
- A repeated key with a different request body is rejected.
- Optimistic concurrency uses resource version/ETag semantics where simultaneous edits are possible.
- Verification and promotion must be bound to an exact candidate version.
- Retrying a promotion request must not execute the cloud change twice.
- Ambiguous external outcomes enter reconciliation rather than automatic duplicate execution.

12. Pagination, Filtering, Sorting, and Search

Capability	Contract
Pagination	Cursor-based; opaque cursor.
Limit	Server-enforced maximum; client may request smaller page.
Filtering	Explicit documented fields only.
Sorting	Allow-listed fields only.
Search	Normalized search over supported fields; no arbitrary query language in MVP.
Consistency	Collection may expose snapshot/freshness metadata where derived.

Pagination cursors must not encode authorization assumptions that survive tenant or permission changes. The server re-validates access when resolving a cursor.

13. Cloud Connection API

Method	Endpoint	Purpose
GET	/api/v1/cloud-connections	List authorized cloud connections.
POST	/api/v1/cloud-connections	Create a connection.
GET	/api/v1/cloud-connections/{id}	Retrieve connection metadata.
PATCH	/api/v1/cloud-connections/{id}	Update non-secret connection metadata/configuration.
POST	/api/v1/cloud-connections/{id}/test	Validate connection.
POST	/api/v1/cloud-connections/{id}/discover	Start discovery.
DELETE	/api/v1/cloud-connections/{id}	Disable/remove connection according to retention policy.

Secrets are never returned by GET or PATCH responses. Connection testing must use stored secret references and return only safe diagnostics.

14. Cloud Scope and Discovery API

Method	Endpoint	Purpose
GET	/api/v1/cloud-connections/{id}/scopes	List discovered/configured scopes.
GET	/api/v1/assets	List assets with filters.
GET	/api/v1/assets/{id}	Retrieve canonical asset.
GET	/api/v1/assets/{id}/observations	Retrieve observations.
GET	/api/v1/assets/{id}/snapshots	Retrieve configuration snapshots.
GET	/api/v1/assets/{id}/dependencies	Retrieve related dependency edges.
POST	/api/v1/cloud-scopes/{id}/discover	Start scoped discovery.

Discovery returns a Job resource. Asset identity is server-derived from canonical provider identifiers; display names cannot be used as identity keys.

15. Security Findings API

Method	Endpoint	Purpose
GET	/api/v1/findings	List/filter findings.
GET	/api/v1/findings/{id}	Retrieve finding.
POST	/api/v1/findings/{id}/analyze	Start/recompute analysis.
POST	/api/v1/findings/{id}/acknowledge	Record acknowledgment if supported by policy.
GET	/api/v1/findings/{id}/risk	Retrieve risk assessment.
GET	/api/v1/findings/{id}/evidence	Retrieve associated evidence.

Finding state is controlled by domain lifecycle rules. Clients cannot directly set arbitrary lifecycle status.

16. Risk API

Method	Endpoint	Purpose
GET	/api/v1/risk/assets/{asset_id}	Current risk view.
GET	/api/v1/risk/findings/{finding_id}	Finding risk.

Method	Endpoint	Purpose
POST	/api/v1/risk/assessments	Request risk assessment.
GET	/api/v1/risk/assessments/{id}	Retrieve assessment.

Risk calculations should expose score, severity/category, contributing factors, freshness, and methodology/policy version where appropriate.

17. Dependency and Impact API

Method	Endpoint	Purpose
GET	/api/v1/assets/{id}/dependencies	Direct/filtered dependencies.
GET	/api/v1/dependencies	Search dependency edges.
POST	/api/v1/dependency-analysis	Run dependency analysis.
POST	/api/v1/impact-analyses	Create impact analysis for an exact candidate version.
GET	/api/v1/impact-analyses/{id}	Retrieve impact analysis.
GET	/api/v1/impact-analyses/{id}/affected-assets	Retrieve affected assets.

Impact analysis must distinguish direct, transitive, and unknown/uncertain impact. Unknown blast radius must not be represented as a clean bounded result.

18. AI API

Method	Endpoint	Purpose
POST	/api/v1/ai/requests	Create an AI request.
GET	/api/v1/ai/requests/{id}	Retrieve AI request metadata/status.
GET	/api/v1/ai/recommendations/{id}	Retrieve validated recommendation.
POST	/api/v1/ai/recommendations/{id}/candidate	Convert eligible recommendation into candidate.
POST	/api/v1/ai/explanations	Request grounded explanation.
POST	/api/v1/ai/remediation-suggestions	Request remediation suggestion.

- AI request context is assembled server-side.
- Client-supplied context is treated as untrusted input and cannot override authoritative cloud observations.
- Structured output is validated before becoming a domain recommendation.
- AI provider details are abstracted from clients.
- Raw model output is not a trusted product state.
- AI requests must respect tenant, data-classification, and provider-boundary policies.

19. Remediation Candidate API

Method	Endpoint	Purpose
GET	/api/v1/candidates	List candidates.
POST	/api/v1/candidates	Create candidate from authorized input/recommendation.
GET	/api/v1/candidates/{id}	Retrieve candidate.

Method	Endpoint	Purpose
POST	/api/v1/candidates/{id}/versions	Create next candidate version when allowed.
GET	/api/v1/candidates/{id}/versions	List versions.
POST	/api/v1/candidates/{id}/submit	Submit exact version for impact/verification.
POST	/api/v1/candidates/{id}/cancel	Cancel candidate.
GET	/api/v1/candidates/{id}/evidence	Retrieve evidence.

Candidate content becomes immutable for the verification/promotion path. Any material change creates a new version and invalidates dependent verification/approval state.

20. DAIL Verification API

Method	Endpoint	Purpose
POST	/api/v1/candidates/{candidate_id}/verification-runs	Start deterministic verification.
GET	/api/v1/verification-runs/{id}	Retrieve verification run.
GET	/api/v1/verification-runs/{id}/results	Retrieve checks/results.
POST	/api/v1/verification-runs/{id}/reconcile	Start reconciliation for an ambiguous external outcome.

- Verification operates on an immutable candidate version.
- Verification execution returns a Job reference where long-running.
- A verification result includes check identity, expected state, observed state, result, evidence reference, timestamp, and verifier/version metadata.
- Partial success cannot satisfy a required complete verification gate.
- Verification failure blocks promotion unless a documented governance rule explicitly changes the candidate path.

21. Approval and Governance API

Method	Endpoint	Purpose
GET	/api/v1/approvals	List approval requests available to caller.
GET	/api/v1/approvals/{id}	Retrieve approval request.
POST	/api/v1/approvals/{id}/approve	Approve exact candidate/version.
POST	/api/v1/approvals/{id}/reject	Reject exact candidate/version.
GET	/api/v1/policies	List policies.
POST	/api/v1/policies/evaluate	Evaluate policy against a proposed action.

Approval must be bound to the exact candidate version, relevant impact result, verification result, and policy context. Changes invalidate approval where required.

22. Promotion API

Method	Endpoint	Purpose
POST	/api/v1/candidates/{id}/promotions	Request governed promotion.

Method	Endpoint	Purpose
GET	/api/v1/promotions/{id}	Retrieve promotion status.
POST	/api/v1/promotions/{id}/reconcile	Reconcile ambiguous external outcome.
GET	/api/v1/promotions/{id}/evidence	Retrieve promotion evidence.

Promotion is the highest-impact API operation. The server re-checks authorization, candidate version, verification status, approval, policy, dependency assumptions, cloud connection health, and other critical preconditions before execution.

The API must never accept a client field such as status=VERIFIED or status=PROMOTED as proof of state.

23. Trusted State API

Method	Endpoint	Purpose
GET	/api/v1/assets/{id}/trusted-state	Retrieve current trusted state.
GET	/api/v1/assets/{id}/trusted-state/history	Retrieve trusted-state history.
POST	/api/v1/assets/{id}/trusted-state/reconcile	Reconcile when external state is ambiguous.

Trusted state is earned only after successful governed promotion or an explicitly defined authoritative reconciliation event. AI output, user declaration, or unverified observation cannot create trusted state.

24. Evidence and Audit API

Method	Endpoint	Purpose
GET	/api/v1/evidence	Search authorized evidence.
GET	/api/v1/evidence/{id}	Retrieve evidence metadata.
GET	/api/v1/evidence/{id}/download	Obtain authorized time-limited artifact access where permitted.
GET	/api/v1/audit-events	Retrieve authorized audit records.
GET	/api/v1/audit-events/{id}	Retrieve audit event.

- Evidence is immutable or append-only according to evidence class.
- Audit events record actor, operation, target, result, correlation ID, and relevant state/version references.
- Sensitive payloads are minimized in audit events.
- Evidence access itself is auditable.

25. Job API

Method	Endpoint	Purpose
GET	/api/v1/jobs	List authorized jobs.
GET	/api/v1/jobs/{id}	Retrieve job state.
POST	/api/v1/jobs/{id}/cancel	Request cancellation when supported.
GET	/api/v1/jobs/{id}/events	Retrieve job progress/events.

Job State	Meaning
QUEUED	Accepted and waiting.
RUNNING	Worker processing.
SUCCEEDED	Completed successfully.
FAILED	Completed with failure.
CANCEL_REQUESTED	Cancellation requested.
CANCELLED	Cancelled safely.
RECONCILIATION_REQUIRED	External outcome is ambiguous.
EXPIRED	No longer valid under retention/timeout policy.

26. Reporting API

Method	Endpoint	Purpose
GET	/api/v1/reports	List reports.
POST	/api/v1/reports	Create report job.
GET	/api/v1/reports/{id}	Retrieve report metadata/status.
GET	/api/v1/reports/{id}/download	Obtain authorized report access.

Large reports are generated asynchronously and stored as controlled artifacts rather than streamed through long-running synchronous requests.

27. Experiment and Evaluation API

Method	Endpoint	Purpose
GET	/api/v1/experiments	List authorized experiments.
POST	/api/v1/experiments	Create experiment definition.
GET	/api/v1/experiments/{id}	Retrieve experiment.
POST	/api/v1/experiments/{id}/runs	Start evaluation run.
GET	/api/v1/evaluation-runs/{id}	Retrieve evaluation run.

Experiment endpoints must not silently affect production trusted state. Production-impacting experiments require explicit environment and policy controls.

28. Webhook/Event Delivery

MVP may expose outbound webhook delivery for selected lifecycle events. Webhooks are notifications, not authoritative commands.

Event	Example
discovery.completed	Discovery job completed.
finding.created	New finding detected.
candidate.created	Remediation candidate created.
verification.completed	Verification run completed.
approval.required	Approval requested.
promotion.completed	Governed promotion completed.
promotion.failed	Promotion failed.
trusted_state.changed	Trusted state changed.
drift.detected	Trusted state no longer matches observed state.

- Webhook payloads contain event ID and resource references.
- Consumers must treat delivery as at-least-once.
- Event IDs support deduplication.
- Webhook signing is required before production external integrations.
- Sensitive data is minimized in webhook payloads.

29. State Transition API Rules

Transition	API Rule
DRAFT →READY_FOR_IMPACT	Explicit submit/prepare command.
READY_FOR_IMPACT →IMPACTED	Server-created impact analysis.
IMPACTED →AWAITING_VERIFICATION	Only after required impact gates.
AWAITING_VERIFICATION →VERIFYING	Verification command.
VERIFYING →VERIFIED	Only deterministic verifier may establish this state.
VERIFIED →APPROVAL_REQUIRED/APPROVED	Policy determines governance path.
APPROVED →PROMOTING	Promotion command after server-side rechecks.
PROMOTING →PROMOTED	Only successful governed execution.
PROMOTED →TRUSTED	Trusted state update after authoritative success.
Any protected state →INVALIDATED	Triggered by material dependency/context/candidate drift.

Clients issue commands; the domain service owns resulting state.

30. Request Validation

- Validate JSON shape, required fields, enum values, string lengths, identifier formats, and nested structures.
- Validate semantic relationships such as candidate/resource/scope compatibility.
- Reject unknown high-risk fields rather than silently accepting them.
- Normalize identifiers and timestamps before domain processing.
- Apply payload-size limits.
- Validate provider-specific configuration through adapter/service validation rather than leaking provider SDK behavior to the API.

31. API Security Controls

Threat	API Control
Broken object authorization	Server-side tenant/resource authorization on every request.
Privilege escalation	Permission checks separated from UI role assumptions.
Replay/duplicate mutation	Idempotency keys + lifecycle state checks.
Injection	Parameterized persistence + structured validation.
Prompt injection	AI context isolation and guardrail pipeline.
Data exfiltration	Data minimization, field-level authorization, redaction.
DoS/abuse	Rate limits, payload limits, queue quotas.
Credential leakage	No secrets in responses/logs/errors.
Cross-tenant enumeration	Authorization-safe 404/response behavior.
Promotion abuse	Dedicated permission + DAIL verification + policy/approval gates.

32. Rate Limits and Quotas

- Apply per-principal and per-tenant rate limits.
- Apply stricter limits to AI, discovery, verification, and promotion endpoints.
- Apply concurrency limits to long-running jobs.
- Return 429 with retry guidance when safe.
- Quota exhaustion must not partially execute a high-impact operation.
- Promotion requests should be serialized or otherwise concurrency-controlled for the same protected resource.

33. Observability and Audit Requirements

Every API request should be traceable through API →service →worker →external dependency where practical.

Telemetry	Required Fields
Request log	Request ID, correlation ID, route, method, status, latency, principal reference, tenant reference where safe.
Job log	Job ID, service, resource ID, state transition, duration, result.
AI telemetry	Request ID, logical model, latency, token/cost metadata where available, safety outcome.
DAIL telemetry	Candidate version, verification run, check result, evidence reference.
Promotion telemetry	Candidate version, actor/service, policy decision, external operation result, reconciliation status.
Security event	Actor, target, operation, outcome, source, timestamp.

Secrets, access tokens, raw credentials, and unnecessarily sensitive customer payloads must never be logged.

34. API Versioning and Compatibility

- Major API versions are explicit, e.g. /api/v1.
- Adding optional response fields is backward-compatible.
- Adding optional request fields is preferred.
- Removing or changing meaning of fields requires a new major version or documented migration.
- Lifecycle semantics are versioned with domain contracts where a semantic change could affect safety.
- Deprecation includes documentation, telemetry, and a migration period.

35. Internal Service Contracts

Internal services should use typed domain commands/events rather than passing arbitrary JSON between components.

Contract	Required Concepts
DiscoveryCommand	Connection, scope, requested resources, actor/job context.
AnalysisCommand	Asset/snapshot reference, rule/policy version.
AIRequestCommand	Task type, authorized context references, policy, tenant.
CandidateCommand	Source recommendation, target resource, exact desired change, candidate version.

Contract	Required Concepts
ImpactAnalysisCommand	Candidate version, graph version/freshness, scope.
VerificationCommand	Candidate version, required checks, environment/scope.
PromotionCommand	Candidate version, approval, verification, policy context.
EvidenceEvent	Evidence type, source, target, content reference, integrity metadata.
AuditEvent	Actor, operation, target, before/after state references, result.

36. Provider and External API Abstraction

The public API must not expose AWS SDK objects or LLM provider-specific request/response formats as its core contract.

- Cloud providers are accessed through Cloud Integration adapters.
- LLM providers are accessed through the AI Abstraction Layer.
- Provider-specific errors are mapped to stable CloudShield error codes.
- Provider resource IDs may be returned as metadata but do not replace CloudShield stable asset IDs.
- Provider changes should not require breaking the public API.

37. API Data Freshness and Provenance

Security, dependency, risk, and trusted-state responses may be derived from observations taken at different times. The API should make freshness visible where it materially affects interpretation.

Field	Purpose
observed_at	When the underlying fact was observed.
computed_at	When a derived result was produced.
source	System/provider that supplied the fact.
source_version	Version of rule/model/graph/policy used.
freshness_status	FRESH, STALE, UNKNOWN, INVALIDATED where applicable.
evidence_id	Link to supporting evidence.

38. Failure and Degradation Semantics

- If AWS discovery fails, the API reports job failure or partial result explicitly; it does not fabricate clean state.
- If the LLM provider fails, the AI operation may degrade to deterministic explanations or remain pending, but must not silently create unsafe candidates.
- If dependency data is incomplete, impact analysis records uncertainty.
- If verification cannot complete, promotion remains blocked.
- If an external promotion outcome is ambiguous, the promotion enters reconciliation-required state.
- If the database is unavailable, mutation endpoints fail safely rather than executing cloud changes without durable state.

39. API Testing Requirements

Test Class	Minimum Coverage
Contract tests	Request/response schemas and status codes.
Authorization tests	Tenant isolation and permission boundaries.
Lifecycle tests	Valid/invalid state transitions.
Idempotency tests	Duplicate and conflicting keys.
Concurrency tests	Stale version and duplicate promotion behavior.
Security tests	Injection, enumeration, auth bypass, rate limits.
AI safety tests	Prompt injection, malformed output, tool boundary.
DAIL tests	Verification binding, partial failure, promotion blocking.
Integration tests	AWS adapters, database, queue, object storage, LLM adapters.
Resilience tests	Timeouts, retries, provider failures, worker restarts.

40. MVP Endpoint Summary

Domain	Core MVP Endpoints
Connections	cloud-connections, test, discover.
Assets	assets, observations, snapshots, dependencies.
Security	findings, risk.
Dependencies	dependencies, impact-analyses.
AI	ai/requests, recommendations, explanations.
Candidates	candidates, versions, submit.
DAIL	verification-runs, results, reconcile.
Governance	approvals, policies/evaluate.
Promotion	promotions, reconcile.
Trust	trusted-state, history, reconcile.
Evidence/Audit	evidence, audit-events.
Jobs	jobs, events, cancel.
Reports	reports, download.

41. Implementation Sequence

1. Define OpenAPI schemas and shared error/resource types.
2. Implement authentication middleware and tenant context.
3. Implement authorization/policy enforcement.
4. Implement cloud connection and scope APIs.
5. Implement asset/discovery/job APIs.
6. Implement findings/risk/dependency APIs.
7. Implement AI abstraction-facing API contracts.
8. Implement candidate/version lifecycle APIs.
9. Implement impact and DAIL verification APIs.
10. Implement approval and promotion APIs with strongest authorization.
11. Implement evidence/audit APIs.
12. Implement reporting and webhook/event capabilities.
13. Add contract, security, lifecycle, concurrency, and integration tests.
14. Publish API documentation and SDK/client generation artifacts.

42. API Definition of Done

- OpenAPI/API contract is version-controlled.
- All protected endpoints enforce authentication and tenant-aware authorization.
- Resource identifiers are stable and server-owned.
- Lifecycle transitions are command-driven and server-enforced.
- Long-running operations use durable Job resources.
- Retryable side-effecting mutations implement idempotency.
- Optimistic concurrency protects mutable safety-critical resources.
- AI responses are validated before candidate creation.
- DAIL verification is bound to exact candidate versions.
- Promotion performs final server-side safety checks.
- Trusted state cannot be created through arbitrary API status updates.
- Evidence and audit events are generated for security/lifecycle-sensitive operations.
- API errors are structured and safe.
- Rate limits and payload limits are implemented.
- Contract, authorization, security, lifecycle, and resilience tests pass.
- The API does not expose provider-specific implementation details as its canonical product contract.

43. Traceability

Source	API Dependency
00 — Master System Specification	Product capabilities, end-to-end control loop, trust model.
01 — Software Requirements Specification	Functional API requirements, security, reliability, integrations.
02 — System Architecture Specification	Application services, workers, queues, cloud/AI/DAIL boundaries.
03 — Functional Specification	Use cases and lifecycle commands.
04 — Product & UX Specification	UI-facing resource views, jobs, errors, approvals.
05 — Domain & Data Model	Canonical resource model and persistence semantics.
06 — State & Lifecycle	State transitions, invalidation, concurrency, reconciliation.
07 — Identity & Dependency	Stable asset IDs, dependency graph, impact semantics.
08 — Security & Threat Model	Authentication, authorization, tenant isolation, abuse/security controls.
09 — Intelligence & AI Architecture	AI abstraction, structured output, context, evaluation.
10 — LLM Integration	Provider-neutral AI integration and error mapping.
11 — AI Safety & Guardrails	AI safety boundary and non-authoritative model behavior.
12–26 — DAIL subsystem specifications	Verification, evidence, promotion, testing, experiment, DevOps contracts.
27 — AWS & Terraform Specification	Infrastructure deployment, workload identities, queues, data stores, observability.

44. Engineering Decisions

Decision	Rationale
REST-oriented JSON API	Simple, broadly interoperable product/client contract.

Decision	Rationale
URL major versioning	Makes breaking changes explicit.
Cursor pagination	Scales better for large cloud inventories.
Command-driven lifecycle	Prevents arbitrary status mutation.
Async Job resource	Matches discovery, analysis, AI, verification, and reporting workloads.
Idempotent side-effecting operations	Protects against retries and duplicate external changes.
Stable CloudShield IDs	Decouples product identity from provider-specific identifiers.
Provider abstraction	Avoids AWS/LLM SDK leakage into product contracts.
DAIL as subsystem API boundary	Keeps deterministic verification integrated without making DAIL the product.
Promotion rechecks	Prevents stale approval/verification from becoming unsafe execution.
Evidence/audit first-class	Makes actions explainable and governable.

45. Glossary

Term	Definition
API	Application Programming Interface exposed by CloudShield AI.
Resource	Stable product-domain object exposed through the API.
Command	Explicit request to cause a governed domain action/state transition.
Job	Durable representation of asynchronous work.
Candidate Version	Immutable candidate content bound to impact, verification, approval, and promotion.
DAIL	Deterministic AI Infrastructure Lifecycle subsystem.
Promotion	Governed execution of an approved and verified change.
Trusted State	State earned after authoritative successful promotion/reconciliation.
Idempotency Key	Client-provided key used to safely retry a mutation.
Tenant	Product authorization/isolation boundary.
Cloud Scope	Account/region/resource boundary for cloud operations.
Provenance	Metadata identifying source, version, time, and evidence supporting a result.